

《大型平台软件设计实习》

实践报告

项目名称：简化数据库系统的设计与实现

专业班级：2401

学 号：L209240132

姓 名：王志强

实验成绩：

批阅教师：

2026 年 09 月 16 日

《简化数据库系统的设计与实现》

一、实践目的

- 综合运用编译原理、操作系统与数据库系统的相关知识，按照“SQL 编译器 → 页式存储系统 → 简化数据库系统”的顺序循序渐进地实现一个数据库系统，体验“从语言到系统”的完整实现过程。
- 掌握词法分析、语法分析（递归下降法）、语义分析与执行计划生成的基本方法，熟悉 SQL 语句到执行计划的转换流程，培养将编译原理应用于实际工程问题的能力。
- 理解文件系统与页式存储管理的基本原理，掌握缓存管理与页替换策略（LRU / FIFO）的实现方法。
- 在前两部分基础上，独立完成表定义、数据存储、条件查询、数据删除等数据库核心功能，加深对执行引擎、存储引擎与系统目录三者交互机制的理解。
- 通过独立完成设计、编码、调试与测试的全过程，提升工程实现能力、问题排查能力与实验报告撰写能力。

二、实践内容

1. SQL 编译器

针对 SQL 子集完成词法分析、语法分析、语义分析与执行计划生成：

- 词法分析：识别关键字（SELECT/FROM/WHERE/CREATE/TABLE/INSERT/INTO/VALUES/DELETE/INT/VARCHAR/AND 等）、标识符、常量（整数、字符串）、运算符（= > < >= <= <>）、分隔符（（），；.），输出 [种别码, 词素值, 行号, 列号] 形式的 Token 流，遇非法字符或未闭合字符串输出定位到行列号的错误。
- 语法分析：递归下降法，支持 CREATE TABLE、INSERT、SELECT、DELETE 四类语句，构建抽象语法树 (AST)，语法错误提示出错位置与期望符号。
- 语义分析：表/列存在性检查、类型一致性检查（INT/VARCHAR 与实际值是否匹配）、INSERT 列数/列序检查，并维护内存 Catalog。

- 执行计划生成：将 AST 转换为由 CreateTable / Insert / Delete / SeqScan / Filter / Project 等算子构成的树形逻辑执行计划，以 JSON 形式输出。

2. 操作系统页式存储

- 设计固定 4KB 大小、页号全局唯一的页式存储系统，支持页的分配、释放、读写，提供 read_page(page_id) / write_page(page_id, data) 接口。
- 实现页缓存机制，支持 LRU、FIFO 两种替换策略，统计缓存命中率并输出替换日志，提供 get_page(page_id) / flush_page(page_id) 接口。

3. 数据库系统

- 执行引擎：解释执行逻辑执行计划，实现 CreateTable、Insert、Delete、SeqScan、Filter、Project 等算子。
- 存储引擎：调用页式存储接口，实现记录 (Row) 与页 (Page) 的序列化/反序列化及表-页映射。
- 系统目录：维护表名、列名、列类型等元数据，系统目录本身作为一张特殊的表进行存储和管理。
- 提供命令行界面 (CLI)，支持 CREATE TABLE、INSERT、SELECT (含 WHERE 条件)、DELETE 四类核心操作，并保证数据持久化。

三、实践方法

整个系统使用 Python 3 标准库独立实现，未引入任何第三方 SQL/数据库框架，以便完整呈现“词法 → 语法 → 语义 → 计划 → 执行 → 存储”的链路，核心设计方法如下：

1. 词法/语法/语义分析

- 词法分析器 (Lexer) 采用手写扫描方式，按字符逐一扫描并结合最长匹配原则切分运算符，行号/列号随扫描过程实时更新，供错误定位使用。
- 语法分析器 (Parser) 为每类语句设计一个 parse_xxx 方法 (parse_create_table / parse_insert / parse_select / parse_delete)，通过 _expect() 统一处理符号匹配与出错信息生成。
- 语义分析器基于内存 Catalog 对 AST 做存在性、类型一致性、列数/列序检查；Catalog 与磁盘上的 pg_catalog 表保持同步，重启后由 pg_catalog 重建。

2. 执行计划与执行引擎

执行计划采用嵌套字典表示算子树（可直接 `json.dumps` 输出，等价于 S-表达式），执行引擎按照 Volcano 迭代器模型对计划树做自顶向下的递归解释执行：Project 从 Filter 取行做投影，Filter 从 SeqScan 取行做条件过滤，SeqScan 顺序扫描表的全部数据页。

3. 页式存储与缓存

- 页大小固定为 4096 字节，页 0 为全局头页，保存总页数、空闲页链表（供页释放后重用）以及系统目录 `pg_catalog` 当前占用的页号列表。
- 每个数据页开头 4 字节记录 `free_offset`（下一条记录的写入位置），记录以 [墓碑标记 1B][长度 2B][payload] 的变长格式顺序追加；INT 编码为 4 字节大端整数，VARCHAR 编码为 2 字节长度前缀加 UTF-8 字节。
- BufferPool 基于 OrderedDict 实现，采用写穿透 (write-through) 策略：写操作立即落盘并更新缓存，从而保证任意时刻的数据安全；LRU 策略在命中时将页移至队尾，FIFO 策略严格按装入顺序淘汰，缓存满时从队首淘汰并记录替换日志。
- DELETE 采用逻辑删除（将记录的墓碑字节置 1），SeqScan 自动跳过被标记的记录，避免实现复杂的页内数据紧缩。

4. 系统目录的自持久化

`pg_catalog` 与普通用户表使用完全相同的记录存储机制（相同的页格式、相同的 `encode_row/decode_row`），因此“系统目录本身也是一张表”这一要求得以直接满足；唯一的特殊处理是 `pg_catalog` 自身当前占用的页号列表保存在磁盘头页中，而不是保存在 `pg_catalog` 内部，从而避免了目录自描述自身位置所带来的循环引用问题。数据库每次启动时会扫描 `pg_catalog` 占用的全部页面，重建内存 Catalog（保留每张表最后一次快照记录，以支持表结构随插入而扩张页面时的更新）。

四、实践步骤

- 1. 规则定义：确定 SQL 子集的关键字表、运算符/分隔符集合，定义 AST 节点 (ast_nodes.py) 与 Catalog 数据结构 (catalog.py)。
- 2. 实现词法分析器 (sql_compiler/lexer.py)，并编写单元测试 (tests/test_lexer.py) 验证关键字/标识符/常量/运算符识别及非法字符、未闭合字符串的报错。
- 3. 实现语法分析器 (sql_compiler/parser.py)，并编写单元测试 (tests/test_parser.py) 覆盖 CREATE TABLE / INSERT / SELECT / DELETE 四类语句及缺分号等语法错误场景。
- 4. 实现语义分析器 (semantic.py)：表/列存在性、类型一致性、列数/列序检查，维护并更新 Catalog。
- 5. 实现执行计划生成器 (planner.py)，将 AST 转换为 CreateTable/Insert/Delete/SeqScan/Filter/Project 构成的树形 JSON 计划。
- 6. 实现页式存储系统 (storage/disk_manager.py)：页分配/释放/读写、全局头页设计 (空闲页链表 + 目录页列表)。
- 7. 实现缓存层 (storage/buffer_pool.py)：LRU/FIFO 替换策略、命中率统计、替换日志。
- 8. 实现存储引擎 (storage_engine.py) 与记录序列化模块 (row.py)：表-页映射、pg_catalog 自持久化、记录的追加写入与逻辑删除。
- 9. 实现执行引擎 (executor.py)：CreateTable/Insert/Delete/SeqScan/Filter/Project 六类算子的具体执行逻辑。
- 10. 将各模块整合为 Database 类 (database.py) 与命令行入口 (main.py)，支持交互式 REPL 与批处理 SQL 文件两种运行方式。
- 11. 编写集成测试 (tests/test_integration.py) 与综合演示脚本 (demo.py)，对标准测试语句、七类典型错误、条件查询与删除、重启后数据持久性、大量数据下的分页与缓存行为进行系统性验证。

五、实践结果

1. 标准测试语句的完整编译流程

以指导书给出的标准语句为例，系统在 verbose 模式下依次输出 Token 流、AST、语义检查结果与执行计划。以 “SELECT id,name FROM student WHERE age > 18;” 为例，词法分析输出的部分 Token 如下：

[KEYWORD, 'SELECT', 1, 1]	[IDENTIFIER, 'id', 1, 8]	[DELIMITER, ',', 1, 10]
[IDENTIFIER, 'name', 1, 11]	[KEYWORD, 'FROM', 1, 16]	[IDENTIFIER, 'student', 1, 21]
[KEYWORD, 'WHERE', 1, 29]	[IDENTIFIER, 'age', 1, 35]	[OPERATOR, '>', 1, 39]
[CONST_INT, '18', 1, 41]	[DELIMITER, ';', 1, 43]	[EOF, '', 1, 44]

对应生成的执行计划 (Project → Filter → SeqScan 三层算子树)：

{ "op": "Project", "columns": ["id", "name"], "child": { "op": "Filter", "predicate": [{ "column": "age", "op": ">", "value": 18 }], "child": { "op": "SeqScan", "table": "student" } } }
--

执行结果：{ 'rows': [{ 'id': 1, 'name': 'Alice' }] }，与预期一致。

2. 错误场景验证

按照指导书要求，对缺分号、列名拼写错误、类型不匹配、值个数不一致、字符串未闭合、重复建表、非法字符共 7 类典型错误逐一测试，系统均能在正确阶段 (词法/语法/语义) 给出准确的错误类型与位置信息，节选如下：

SELECT id FROM student	-> [语法错误] 遇到 '<EOF>' (行:1,列:23)，期望符号: ;
SELECT idd FROM student;	-> [语义错误] 类型:列不存在，位置:表 student 列 idd
INSERT ... VALUES ('x','Bob',20);	-> [语义错误] 类型:类型不匹配，位置:列 id
INSERT ... (id,name,age) VALUES(2,'Bob')	-> [语义错误] 类型:列数不匹配，目标列数 3 与值数 2 不一致
INSERT ... VALUES (3,'Bob,20);	-> [词法错误] 字符串未闭合 (行:1,列:44)
CREATE TABLE student(id INT);	-> [语义错误] 类型:表已存在，位置:表 student
... WHERE id # 1;	-> [词法错误] 非法字符 '#' (行:1,列:33)

3. 条件查询、删除与再查询

插入 5 条记录后，SELECT * FROM student WHERE age > 18 正确返回 3 条满足条件的记录；执行 DELETE FROM student WHERE id = 1 后返回“删除 1 行”，随后 SELECT * FROM student 的结果中 id=1 的记录已不存在，其余 4 条记录保持不变，验证了逻辑删除机制的正确性。

4. 数据持久性验证

关闭数据库连接 (`Database.close()` , 触发 `BufferPool` 落盘与磁盘头页写回) 后 , 重新创建 `Database` 实例指向同一数据库文件 , 执行 `SELECT * FROM student` , 返回的结果与关闭前完全一致 , 验证了表结构与数据记录均已正确持久化 , 程序“重启”后数据不丢失。

5. 大量数据下的分页与缓存行为

创建表 `t(id INT, name VARCHAR)` 并连续插入 600 行数据 , 同时将 `BufferPool` 容量刻意设置为 3 页以便观察替换行为 , 实测结果如下 :

```
插入 600 行后, 表 t 占用的数据页数量: 3 -> 页号: [2, 3, 4]
缓存统计 (buffer_capacity=3, policy=LRU): {'hits': 606, 'misses': 0, 'hit_rate': '100.00%'}
替换日志: [LRU] 缓存已满, 淘汰页 2, 装入页 4
SeqScan 校验: 实际扫描到 600 行 (应等于插入行数 600)
```

结果表明 : 随着数据增长 , 存储引擎能够正确地按需分配新页 (本例中数据被组织到 2、3、4 三个连续数据页中) ; 由于写穿透策略下页面在创建时即进入缓存 , 命中率保持 100% , 未命中数为 0 ; 当活跃页数超过缓存容量时 , LRU 策略正确淘汰了最久未被访问的页 (页 2) , 符合预期。

6. 单元测试与集成测试

使用 `python -m unittest discover -s tests -v` 运行全部测试 , 覆盖词法分析 (6 项) 、语法分析 (8 项) 、集成场景 (7 项 , 含建表/插入/查询、重复建表报错、类型不匹配报错、列数不匹配报错、删除后再查询、重启持久性、批量插入多页) , 全部 21 项测试通过 :

```
Ran 21 tests in 0.022s
OK
```

六、实践结论

本次实践按照“SQL 编译器 → 页式存储系统 → 简化数据库系统”的顺序完整实现了指导书要求的核心功能 : 词法分析器能够正确切分 Token 流并定位词法错误 ; 语法分析器

基于递归下降法能够正确构建四类语句的 AST 并给出准确的语法错误提示；语义分析器能够正确执行存在性、类型一致性与列数/列序检查；执行计划生成器能够将 AST 转换为清晰的树形算子计划。页式存储系统实现了固定大小页的分配/释放/读写，缓存层支持 LRU/FIFO 两种替换策略并能正确统计命中率、记录替换日志。在此基础上构建的简化数据库系统能够正确执行 CREATE TABLE、INSERT、SELECT（含 WHERE 条件）、DELETE 四类核心语句，系统目录以“特殊表”的形式自持久化，且经测试验证程序重启后数据不丢失。

从实测结果看，系统在正确性方面表现良好：21 项单元/集成测试全部通过，标准测试语句与全部 7 类错误场景均按预期给出结果，600 行规模下的分页与缓存替换行为也符合设计预期。同时，本实现在若干地方做了工程上的简化取舍：例如缓存层采用写穿透而非“脏页延迟写回”，牺牲了一定的写性能以换取实现简单和数据安全；WHERE 子句仅支持 AND 连接的简单比较条件，未实现 OR、括号嵌套及 UPDATE/JOIN/ORDER BY/GROUP BY 等指导书中的可选扩展项。总体而言，本实践较为完整地贯通了编译原理、操作系统与数据库系统三方面的核心知识点，达到了“从语言到系统”的实践目标。

七、实践小结

通过本次实践，我对“一条 SQL 语句从输入到得到结果”的完整链路有了更加具体的认识：以往在课堂上分别学习编译原理、操作系统与数据库课程时，词法分析、页式存储、查询执行这些概念相对独立；而在本次实践中把它们串联为一个可运行的系统后，才真正体会到执行计划是连接“编译前端”和“存储后端”的关键纽带，也理解了为什么教材中反复强调 Catalog（系统目录）在数据库系统中的核心地位。

实现过程中遇到的主要困难及解决方式：一是系统目录的“自举”问题——pg_catalog 本身也需要通过页式存储来持久化，但它又需要知道自己占用了哪些页，容易出现循环依赖；最终通过把 pg_catalog 自身的页号列表单独保存在磁盘头页中解决了这一问题。二是变长记录跨页写入的处理——当一页剩余空间不足以容纳下一条记录时，需

要正确地分配新页并更新表的页号列表，同时还要同步更新 `pg_catalog` 中的快照记录，调试时通过编写小规模单元测试逐步验证记录的编码/解码是否自治。三是缓存替换策略的验证——最初很难直观地判断 LRU 是否生效，后来通过刻意把缓存容量设置得很小（如 3 页）并结合替换日志输出，才能清楚地观察到页面被淘汰和重新装入的过程。

这次实践也让我意识到自己在工程实现的“边界取舍”方面还有提升空间，例如是否要支持脏页延迟写回、是否要实现真正的空闲空间管理与页内紧缩等，这些在本次实践中都做了简化处理。后续如果继续完善这个系统，我希望能够补充 UPDATE 语句、简单的嵌套循环 JOIN、基于索引的查找，并将当前“写穿透”的缓存策略改造为更贴近真实数据库的“脏页延迟写回 + WAL 日志”机制，以进一步加深对数据库系统底层实现的理解。